Implementation — Smart Choices

Six compact patterns that keep the controller safe, correct, and timing-aware

Andrei

1 Boundary-safe routing

```
      if (rangeOverflows(addr, bytes))
  return false;
bool first = isRomAddress(addr);
bool last = isRomAddress(addr + bytes - 1);
return first == last;
```

Reject early: overflow and ROM/RAM-crossing requests.

2 Per-byte protection

```
      for (uint32_t i = 0; i < bytes; ++i) {
  uint8_t owner = getOwner(addr + i);
  if (owner != 255 &&
      owner != user) return false;
}
```

Correct at edges: wide accesses may touch two blocks.

3 Read–modify–write

```
      uint32_t old = memoryRead(addr);
uint32_t next =
  (old & 0xFFFFFF00u) |
  (data & 0xFFu);
memoryWrite(addr, next);
```

Preserve data: change one byte, keep the other three.

4 Latency-independent handshake

```
      mem_r.write(true);
do {
  wait();
  wait(SC_ZERO_TIME);
} while (!mem_ready.read());
```

Protocol-driven: wait for ready, not a guessed latency.

5 Zero-latency ROM

```
      for (uint32_t i = 0;
    i < latencyRom; ++i)
  wait();
rdata.write(romRead(addr, wide));
finishSuccess();
```

No special case: latency 0 simply performs no wait.

6 Little-endian byte storage

```
      uint8_t value =
  it == memory.end() ? 0 : it->second;
result |= static_cast<uint32_t>(value)
       << (i * 8);
```

Byte-addressable: naturally supports unaligned access.

Validate early · protect every touched byte · preserve untouched data · synchronize through signals